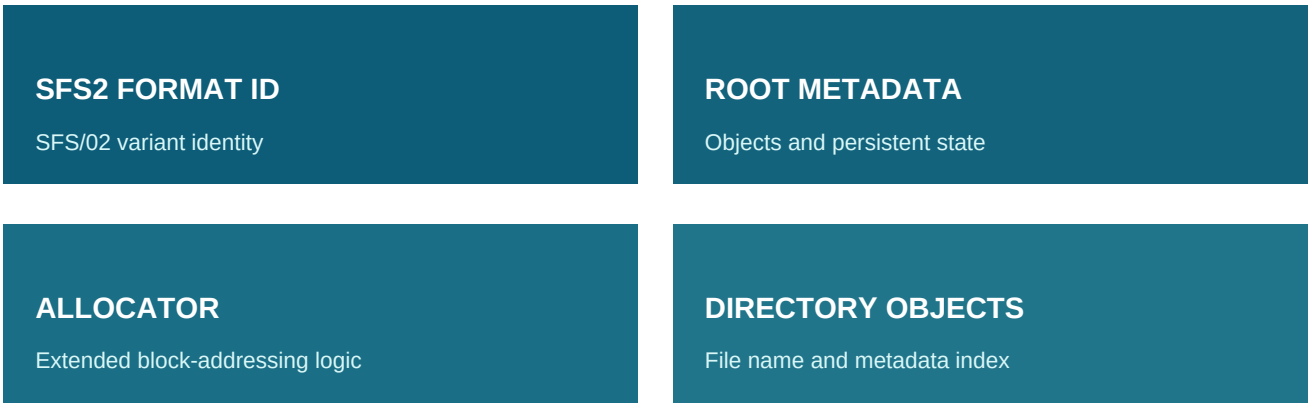
SmartFileSystem 2 - Amiga SFS2

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

The later Amiga SmartFileSystem format, distinguished by the SFS\02 DOS type and different capacity and addressing expectations from SFS0.

This guide documents the original format's technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

SmartFileSystem 2 - Amiga SFS2

HISTORICAL CHRONOLOGY

SmartFileSystem originated as an Amiga alternative to the classic OFS and Fast File System on-media designs. The AmigaOS 4 ecosystem distinguishes SmartFileSystem 2 as a later variant, not merely a different package name for the original SFS0. Its filesystem capability table lists distinct DOS types, capacities and compatibility limits for SFS and SFS2.

SFS2 continued the effort to support larger disks and contemporary AmigaOS workflows while preserving the SmartFileSystem focus on responsiveness and crash resilience. Historical source documentation and AmigaOS release archives are required when identifying actual revisions. A general description of SFS is insufficient for a driver that must parse the SFS2 disk layout.

SmartFileSystem 2 - Amiga SFS2

ORIGINAL ON-DISK ARCHITECTURE

The SFS2 volume uses its own SFS variant identity and persistent root, object and allocation metadata. It is not AmigaDOS FFS DOS1/3/5, and an SFS0 module cannot be assumed to parse its structures. Addressing fields and volume size boundaries must be selected according to the SFS2 version and physical media geometry.

The format operates through recorded object identities and storage allocation structures that support indexed directory lookup. Historical SmartFileSystem source uses specialised object and transaction mechanisms. Even if code sharing is possible between SFS versions, a safe interpreter must explicitly validate the format variant and its metadata invariants.

SmartFileSystem 2 - Amiga SFS2

ALLOCATION AND DIRECTORIES

Directory entries associate names with persistent file objects. Allocation state must be coordinated with namespace changes to avoid free blocks still referenced by file metadata. SFS2 targets larger practical storage sizes than the earlier SFS0 mode; arithmetic should use appropriately wide addresses and account for the format’s documented limits.

The exact file and directory record layout must be derived from source and AmigaOS documentation rather than inferred from generic DOS semantics. Name handling, file comments, ownership and timestamps are affected by AmigaOS APIs and filesystem capabilities. Deletion, rename and extension of files must maintain the filesystem’s object and free-space invariants.

SmartFileSystem 2 - Amiga SFS2

INTEGRITY AND RECOVERY

SmartFileSystem’s transaction design attempts to maintain a consistent metadata state after interruptions, but the specific log or transaction mechanism and its scope depend on the format revision. SFS2 should not be described as identical to ext3 journalling or copy-on-write ZFS. Only verified source rules and interruption tests justify recoverability claims.

Recovery requires consistent root and transaction state, object references, directory index and allocation map. Cross-referencing them avoids reallocating blocks that remain reachable. A reader that sees the SFS2 signature may still be unable to interpret the version’s metadata; in that case it must remain read-only and refuse repairs.

SmartFileSystem 2 - Amiga SFS2

DESIGN RATIONALE AND LIMITATIONS

The motivation is modern Amiga performance and capacity without the severe structural constraints of OFS and FFS. SFS2's independent DOS identity protects compatibility by allowing old handlers to reject unknown disks rather than silently misinterpret newer metadata.

Historical capability tables are implementation-specific: an operating-system handler's advertised maximum file size does not automatically define every possible on-media theoretical limit. Interoperability with legacy 68K, AmigaOS4 and foreign platforms requires separate evidence.

SmartFileSystem 2 - Amiga SFS2

INFILTRATOROS INTEGRATION BOUNDARY

SFS2.so must not be a copy of SFS.so with a new filename. Tests must show independent SFS2 media detection, address width, object records and valid recovery behaviour. A successful shared-library export check does not establish those properties.

Our packaged Linux adapter, when present, must be qualified on exact kernel versions. Filesystems and Defragmenter are consumers of this separate module, not owners of a parallel SmartFileSystem implementation.

SmartFileSystem 2 - Amiga SFS2

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. AmigaOS official filesystem matrix distinguishing SFS0 and SFS2

https://wiki.amigaos.net/wiki/UserDoc%3AAmigaOS_File_Systems

2. SmartFileSystem historical source package and changes

https://os4depot.net/?file=driver%2Ffilesystem%2Fsfs.lha&function=showcontent

3. Amiga SmartFileSystem background reference

https://wiki.amiga.org/index.php?title=SmartFileSystem

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

SFS2 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Recognise the SFS2 DOS type separately from earlier SmartFileSystem variants; format identity is a prerequisite for every subsequent structure read.

2. Read the volume root and persistent metadata with SFS2's version-specific addressing and structure limits rather than borrowing SFS0 constants.

3. Trace file and directory objects through the current object-index representation. Check keys and object types before interpreting their payloads.

4. Resolve space ownership through SFS2 allocation metadata. Filesystem-level volume sizes and physical device offsets are distinct quantities.

5. Inspect the transaction or committed-root state before deciding which version of a modified directory or extent map is authoritative.

6. Validate every block and extent range against media geometry, including integer-width transitions that motivated newer SFS generations.

7. Match directory ordering and name rules to the actual media format rather than guessing from the visible AmigaDOS API surface.

8. Use the original SFS source revision to settle disputed structure details; sharing an application interface does not imply identical on-disk bytes.

SFS2 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Reject media with a format identifier inconsistent with the decoder’s selected SFS generation, even if some root fields appear plausible.

2. Check that all live object references can be resolved and that no two incompatible owners claim the same allocation units.

3. Do not equate a valid transaction marker with successful validation of all children, directory records or extent metadata.

4. Interrupted writes can leave multiple candidate states. Preserve the media and identify a consistent committed root before any repair.

5. Respect higher-capacity addressing boundaries; a 32-bit cast of a wider physical or logical address can point at an unrelated block.

6. Cross-version source compatibility is not the same as disk compatibility. SFS0 and SFS2 need independent qualification evidence.

7. Unknown optional features and inconsistent metadata should be explicitly reported instead of silently normalised or reformatted.

8. Keep adapter-specific tests, release claims and Linux kernel compatibility distinct from the original format’s historical technical reference.